



CHAPTER 4

TIME AND SPACE COMPLEXITY

Learning Outcomes

At the end of this lecture, you should be able to explain:

- ▶ Empirical Analysis
- ▶ Time Complexity
 - ▶ Measuring Operations as a Function of Input Size
- ▶ Space Complexity
 - ▶ Space/Time Tradeoff

Time and Space Complexity

- ▶ Sometimes, there are more than one way to solve a problem.
- ▶ We need to learn how to compare the performance different algorithms and choose the best one to solve a particular problem.
- ▶ **Time** and **Space complexity** are different aspects of analyzing the efficiency of an algorithm.
- ▶ Time and space complexity depends on lots of things like hardware, operating system, processors, etc.
- ▶ However, we don't consider any of these factors while analyzing the algorithm.
- ▶ We will only consider the execution time of an algorithm and the storage space required by the algorithm.

Time complexity

- is a measure of execution time using the Big-O notation
- estimated by counting the number of elementary steps performed by any algorithm to finish execution
- algorithm's performance may vary with different types of input data, hence for an algorithm, we usually use the worst-case Time complexity of an algorithm because that is the maximum time taken for any input size.

Space complexity

- measures the amount of memory space used by an algorithm. The memory space considered is the main memory.
- whenever a program executes, the program is to be loaded on the primary memory. Then during execution, how much space will be occupied will be decided by the space complexity.
- the number of instruction getting executed or the number of instruction residing in the algorithm will decide the space complexity.

Empirical analysis

Empirical Approach

- ▶ Run two competing implemented algorithms
- ▶ Collect the running time required by each algorithm to complete
- ▶ Perform the experiments for both algorithms over an arbitrary number of times
- ▶ Calculate and compare the average running time of both algorithms

Empirical Approach

- ▶ Comparative timing of programs is difficult
- ▶ Subject to experimental errors arising from uncontrolled factors:
 1. System load
 2. Programming language used
 3. Compiler efficiency

Programmer Bias

- ▶ Programmer bias
 - ▶ Implementation of the programs
 - ▶ Will be reflected in the timings captured
- ▶ Code tuning
 - ▶ One program may received more code tuning than the other
 - ▶ Can often reduce the running time of a program

Code Tuning

- ▶ When running times for two programs being compared differ by a constant factor regardless of input size (i.e., their growth rates are equal) then differences in code tuning for each program might affect the running time of each program

Simulation

- ▶ Another approach to analysis is simulation
- ▶ Models the problem with a computer program then run it to get a result
- ▶ Is different from empirical comparison of two competing programs because the purpose of the simulation is to perform analysis of an algorithm for solving difficult and hard problems

Motivation

- ▶ Even a simple program can be noticeably inefficient.
- ▶ A program's efficiency affects its overall performance (e.g. response time) and user satisfaction.
- ▶ How can we measure efficiency so that we can compare the efficiency of various approaches to solve a problem?

Analysis of Algorithms

- ▶ The process of measuring the complexity of an algorithm
- ▶ Types of complexity
 - ▶ Space complexity: the memory required to execute the code
 - ▶ Time complexity: the time the code takes to execute
- ▶ There is usually a trade-off between time and space requirements of a program.

Experimental Studies

- ▶ One way to study the efficiency of an algorithm is to implement it and experiment by running the program on various test inputs while recording the time spent during each execution.
- ▶ E.g. collecting running times in Java is based on the use of the `currentTimeMillis` method of the `System` class:
 - ▶ Record the time immediately before executing the algorithm and then immediately after
 - ▶ Compute the difference between those times to obtain the elapsed time of the algorithm's execution

Code Fragment for Timing an Algorithm in Java

```
1 long startTime = System.currentTimeMillis();  
2 /* (run the algorithm) */  
3 long endTime = System.currentTimeMillis();  
4 long elapsed = endTime - startTime;
```

Note: for extremely quick operations, Java provides the **nanoTime** method that measures in nanoseconds rather than milliseconds

Experimental Method

- ▶ As we are interested in the general dependence of running time on the size and structure of the input, we should perform independent experiments on many different test inputs of various sizes.
- ▶ We can then visualize the results by plotting the performance of each run of the algorithm as a point with x-coordinate equal to the input size n and y-coordinate equal to the running time t .
- ▶ Such a visualization provides some intuition regarding the relationship between problem size and execution time for the algorithm.

Experimental Method (cont'd)

- ▶ This may be followed by a statistical analysis that seeks to fit the best function of the input size to the experimental data.
- ▶ To be meaningful, this analysis requires that we choose good sample inputs and test enough of them to be able to make sound statistical claims about the algorithm's running time.

Challenges of Experimental Analysis

- ▶ Experimental running times of two algorithms are difficult to directly compare unless the experiments are performed in the same hardware and software environments.
- ▶ Experiments can be done only on a limited set of test inputs; hence, they leave out the running times of inputs not included in the experiment (and these inputs may be important)
- ▶ An algorithm must be fully implemented in order to execute it to study its running time experimentally.

Goal for Algorithm Analysis

- ▶ Due to the shortcomings of experimental analysis, we need an approach to analyzing the efficiency of algorithms that:
- ▶ Allows us to evaluate the relative efficiency of any two algorithms in a way that is independent of the hardware and software environment
- ▶ Is performed by studying a high-level description of the algorithm without need for implementation
- ▶ Takes into account all possible inputs

Time Complexity

TIME AND SPACE
COMPLEXITY

Time Complexity

- ▶ The time complexity of algorithms is usually more important than space complexity.
- ▶ The actual time needed for an algorithm is difficult to compute accurately. Therefore,
 - ▶ We estimate the times for the best case, average case and worst case.
 - ▶ We express time efficiency as a factor of the problem size (e.g., when searching a collection of data, the problem size is the number of items in the collection).

Counting Primitive Operations

- ▶ To analyze the running time of an algorithm without performing experiments, analyze the algorithm by counting primitive operations.
- ▶ Example of primitive operations:
 - ▶ Assigning a value to a variable
 - ▶ Following an object reference
 - ▶ Performing an arithmetic operation
 - ▶ Comparing two numbers
 - ▶ Accessing a single element of an array by index
 - ▶ Calling a method
 - ▶ Returning from a method

Counting Primitive Operations (cont'd)

- ▶ Instead of trying to determine the specific execution time of each primitive operation, we will simply count how many primitive operations are executed, and use this number t as a measure of the running time of the algorithm.
- ▶ This operation count will correlate to an actual running time in a specific computer.
- ▶ Thus, the number t of primitive operations an algorithm performs will be proportional to the actual running time of that algorithm.

Measuring Operations as a Function of Input Size

- ▶ To capture the order of growth of an algorithm's running time, we will associate a function $f(n)$ with the algorithm.
- ▶ This function $f(n)$ will characterize the number of primitive operations that are performed as a function of the input size n .

Difficulty of Average-Case Analysis

- ▶ An algorithm may run faster on some inputs than it does on others of the same size. Thus, we may wish to express the running time of an algorithm as the function of the input size obtained by taking the average over all possible inputs of the same size.
- ▶ Unfortunately, such an average-case analysis is quite challenging as it requires us to define a probability distribution on the set of inputs.

Focusing on the Worst-Case

- ▶ Worst-case analysis is much easier than average-case analysis, as it requires only the ability to identify the worst-case input, which is often simple.
- ▶ Also, this approach typically leads to better algorithms due to the focus on designing to cope for better performance in the worst case.

Note: Useful formulas

- ▶ $1 + 2 + \dots + n = n(n + 1) / 2$
- ▶ $1 + 2 + \dots + (n - 1) = n(n - 1) / 2$

Problem: Analyzing the efficiency of algorithms for computing $1 + 2 + \dots n$

28

Algorithm A	Algorithm B	Algorithm C
<pre>sum = 0 for i = 1 <i>to</i> n sum = sum + i</pre>	<pre>sum = 0 for i = 1 <i>to</i> n { for j = 1 <i>to</i> i sum = sum + 1 }</pre>	<pre>sum = n * (n + 1) / 2</pre>

Fig. 4.1: Three algorithms for computing $1 + 2 + \dots n$ for an integer $n > 0$

Analysis of the 3 Algorithms

	Algorithm A	Algorithm B	Algorithm C
Assignments	$n + 1$	$1 + n(n + 1) / 2$	1
Additions	n	$n(n + 1) / 2$	1
Multiplications			1
Divisions			1
Total operations	$2n + 1$	$n^2 + n + 1$	4

Fig. 4.2: The number of operations required by the algorithms for Fig 4.1

(Note: *operations for the **for** loops are ignored for simplicity*)

Analysis of Algorithm A

sum = 0

for i = 1 to n

 sum = sum + i

Total operations

= total assignments + total additions

= (1 assignment for initializing sum + n assignments in
 for loop body) + (n additions in for loop body)

= (1 + n) + (n)

= 2n + 1

Analysis of Algorithm B

```

sum = 0
for i = 1 to n {
  for j = 1 to i
    sum = sum + 1
}

```

Total operations

= total assignments + total additions

= (1 assignment for initializing sum + n assignments in
for loop body) + (n additions in for loop body)

= $(1 + [1+2+3+\dots+n]) + [1+2+3+\dots+n]$

= $(1 + [n(n+1)/2]) + [n(n+1)/2]$

= $n^2 + n + 1$

i	j	Total =	Total +
1	1..1	1	1
2	1..2	2	2
3	1..3	3	3
...	...	...	...
n	1..n	n	n

Analysis of Algorithm C

$$\text{sum} = n * (n + 1) / 2$$

Total operations

= total assignments + total multiplications + total
additions + total divisions

$$= 1 + 1 + 1 + 1$$

$$= 4$$

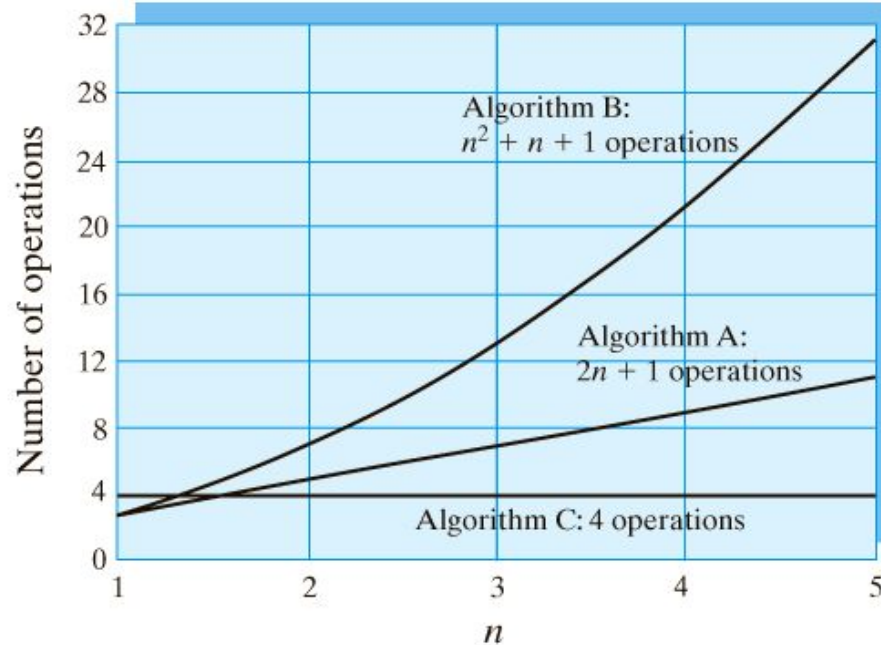


Fig. 2.3: The number of operations required by the algorithms in Fig. 9.1 as a function of n

Expressing Algorithm Efficiency

- ▶ The actual time requirement of an algorithm cannot be computed. Instead,
 - ▶ We find a function of the problem size that represents the algorithm's time requirement.
 - ▶ As the problem size increases by some factor, the value of the function increases by the same factor and this result indicates the increase in the algorithm's time requirement.
- ▶ Computer scientists use the Big-Oh notation to represent an algorithm's efficiency / complexity.

Space Complexity

TIME AND SPACE
COMPLEXITY

Space Complexity

- ▶ Space is the other computing resource that is commonly of concern to programmers.
- ▶ Just as computers have become much faster over the years, they have also received greater allotments of memory.
- ▶ Even so, the amount of available disk space or main memory can be significant constraints for algorithm designers.

Space Complexity

- ▶ The analysis techniques used to measure space requirements are similar to those used to measure time requirements.
- ▶ However, while time requirements are normally measured for an algorithm that manipulates a particular data structure, space requirements are normally determined for the data structure itself.
- ▶ The concepts of asymptotic analysis for growth rates on input size apply completely to measuring space requirements.

Space Complexity

- ▶ A data structure's primary purpose is to store data in a way that allows efficient access to those data.
- ▶ To provide efficient access, it may be necessary to store additional information about where the data are within the data structure.
- ▶ For example, each node of a linked list must store a pointer to the next value on the list.
- ▶ All such information stored in addition to the actual data values is referred to as **overhead**.

Space Complexity

- ▶ Ideally, overhead should be kept to a minimum while allowing maximum access.
- ▶ The need to maintain a balance between these opposing goals is what makes the study of data structures so interesting.

Space/Time Tradeoff

- ▶ One important aspect of algorithm design is referred to as the space/time trade-off principle.
- ▶ The space/time tradeoff principle says that one can often achieve a reduction in time if one is willing to sacrifice space or vice versa.
- ▶ Many programs can be modified to reduce storage requirements by “packing” or encoding information.

Space/Time Tradeoff

- ▶ “Unpacking” or decoding the information requires additional time.
- ▶ Thus, the resulting program uses less space but runs slower.
- ▶ Conversely, many programs can be modified to pre-store results or reorganize information to allow faster running time at the expense of greater storage requirements.
- ▶ Typically, such changes in time and space are both by a constant factor.

Lookup Tables

- ▶ A classic example of a space/time tradeoff is the lookup table.
- ▶ A lookup table pre-stores the value of a function that would otherwise be computed each time it is needed.
- ▶ For example, $12!$ is the greatest value for the factorial function that can be stored in a 32-bit int variable.

Lookup Tables

- ▶ If you are writing a program that often computes factorials, it is likely to be much more time efficient to simply pre-compute the 12 storable values in a table.
- ▶ Whenever the program needs the value of $n!$ for $n \leq 12$, it can simply check the lookup table. (If $n > 12$, the value is too large to store as an int variable anyway.)
- ▶ Compared to the time required to compute factorials, it may be well worth the small amount of additional space needed to store the lookup table.

Lookup Table

- ▶ Lookup tables can also store approximations for an expensive function such as sine or cosine.
- ▶ If you compute this function only for exact degrees or are willing to approximate the answer with the value for the nearest degree, then a lookup table storing the computation for exact degrees can be used instead of repeatedly computing the sine function.
- ▶ Note that initially building the lookup table requires a certain amount of time.
- ▶ Your application must use the lookup table often enough to make this initialization worthwhile.

Space/Time Tradeoff – Binary Sort

- ▶ Another example of the space/time tradeoff is typical of what a programmer might encounter when trying to optimize space.
- ▶ Here is a simple code fragment for sorting an array of integers.

```
for (i=0; i<n; i++)
```

```
    B[A[i]] = A[i];
```

Space/Time Tradeoff – Binary Sort

- ▶ We assume that this is a special case where there are n integers whose values are a permutation of the integers from 0 to $n - 1$.
- ▶ This is an example of a Binary sort.
- ▶ Binary sort assigns each value to an array position corresponding to its value.
- ▶ This is efficient but it also requires two arrays of size n .

Space/Time Tradeoff – Binary Sort

- ▶ Next is a code fragment that places the permutation in order but does so within the same array (thus it is an example of an “in place” sort).

```
for (i=0; i<n; i++)  
    while (A[i] != i) // Swap element A[i] with A[A[i]]  
        DSutil.swap(A, i, A[i]);
```

Space/Time Tradeoff – Binary Sort

- ▶ The second code fragment requires more time to run than the first code fragment.
- ▶ Typically, the second version takes nearly twice as long to run as the first, but it only requires half the space.

Disk-based Space/time Tradeoff

- ▶ A second principle for the relationship between a program's space and time requirements applies to programs that process information stored on disk.
- ▶ The disk-based space/time tradeoff principle is almost the reverse of the space/time tradeoff principle for programs using main memory.

Disk-based Space/time Tradeoff

- ▶ The **disk-based space/time** tradeoff principle states that the smaller you can make your disk storage requirements, the faster your program will run.
- ▶ This is because the time to read information from disk is enormous compared to computation time, so almost any amount of additional computation needed to unpack the data is going to be less than the disk-reading time saved by reducing the storage requirements.
- ▶ Naturally this principle does not hold true in all cases, but it is good to keep in mind when designing programs that process information stored on disk.

END OF CHAPTER 4

TIME AND SPACE COMPLEXITY